

Feature Brainstorm Evaluation

Evaluated against: hackathon-winnability, plausibility with Claude + Flask/SQLite in the time you likely have, and fit with what's already built (Rounds 1-3, Simulator, saved ideas, auth).

Effort scale: **S** = 1-3 hrs, **M** = 4-8 hrs, **L** = 1-2 days, **XL** = multi-day / needs real data infra.

Simulator tab additions

Cash flow statement (in the Simulator)

Verdict: Build it. High value, low risk.

You already compute monthly capital/income series client-side. A cash flow statement is just re-presenting that same math as "cash in / cash out / net" per month instead of two cumulative lines. No new data needed.

Effort: S (2-3 hrs)

EBITDA / margin, break-even point (separate graph)

Verdict: Build it, keep it simple.

Break-even = the month where cumulative capital growth turns positive net of costs — you already have the cost/revenue streams. A single vertical marker on the existing chart ("break-even: month 14") is a 30-minute add. A full EBITDA calculation needs a "fixed costs vs variable costs" split you don't currently collect — worth adding as two optional fields per stream rather than a new subsystem.

Effort: S-M (3-5 hrs)

LLM → "Labor Bureau" lookup that recommends growth-rate percentages by sector

Verdict: Cut for the hackathon. Keep as a v2 idea.

This is the riskiest item on the list. There's no free, structured, reliably-scraped "labor bureau" API you can hit live in a demo (BLS data exists but it's clunky, rate-limited, and not structured for this). Doing it via a second Claude call with web search is *possible* but adds latency, another point of failure, and — critically — you already write "grounded, cited, real sources" into Round 1/3. Cutting corners here with a vague LLM guess undermines that story. A judge will ask "how do you know that growth rate is right?" and "we asked an LLM" is a weak answer for a number that then drives a financial simulation. **If you still want a version of this:** make it a single optional button ("Suggest a growth rate for this sector") that calls Claude with web search, shows the number *with its source URL*, and lets the user accept or ignore it — never auto-applies it silently. That's buildable in the time you have; a live BLS API integration is not.

Effort: M (5-7 hrs) for the capped version. XL for a real labor-bureau integration — skip.

Stop-loss (auto-halt simulation past a threshold)

Verdict: Build it. Cheap, demoable, sounds sophisticated.

Add one input ("Stop simulating if capital drops below \$X"), and in your month-by-month loop, break early and flag the month it happened. Judges like seeing a model that knows its own limits.

Effort: S (1-2 hrs)

Round 2 (business model) additions

Debt vs. equity decision tree

Verdict: Build a simplified version.

A true decision tree UI (branching questions) is more effort than it's worth for a hackathon demo. A "decision tree" that's actually 3-4 conditional rules feeding a single Claude-generated recommendation (similar to your tax structure recommendation) gets 90% of the visual/perceived value for a fraction of the build.

Effort: S-M (3-4 hrs) for the simplified version. L+ for a real interactive branching tree — not worth it.

Industry margin chart (by sector)

Verdict: Build it — but be honest about the data source.

You need a small reference table of typical gross/net margins by sector (5-8 sectors covers most pitches: retail, food service, SaaS/software, professional services, e-commerce, manufacturing, personal services). Hardcode a well-sourced table (cite SBA/industry association figures in a footnote) rather than trying to look it up live — it's static data, doesn't change per-request, and a hardcoded table is **more** trustworthy for a demo than an LLM guess. Store the matched sector name in the submission JSON now — you're right that this pre-classification will make the "labor bureau" feature (if ever built) faster and cheaper, since you won't re-derive sector from scratch each time.

Effort: M (4-6 hrs) — most of that is building/sourcing the reference table.

Enhanced startup costs: payroll, employee headcount, total capital needed vs. budget

Verdict: Build it. This is one of the highest-value items on the whole list.

You already collect budget in Step 0. Add optional employee count + average role/salary fields to Round 2, have Claude fold payroll into costBreakdown, and add one explicit line: "Total needed: \$X vs. your budget: \$Y — you're short by \$Z" or "well within budget." This is the single most "did this app actually help me" moment you can add, and it's mostly using data you already have.

Effort: M (4-6 hrs)

"Reality checkbox": cushion fund / loan amount

Verdict: Build it, and here's how to make it more than a checkbox.

Turn it into 2-3 fields, not a checkbox: (1) cash cushion the founder currently has, (2) how much loan/outside capital they're considering, (3) monthly personal expenses they need covered. Then compute and show: **runway** (cushion ÷ monthly burn), and whether the combination of budget + cushion + loan actually covers the Round 2 startup cost total. This reuses your existing capital-vs-budget math from the item above. **More ideas for what "reality check" can flag:**

- Runway under 3 months → red flag, shown prominently
- Loan amount that would require monthly payments exceeding a rough affordability threshold vs. projected revenue

- No cushion AND no loan AND a Round 1 startup cost estimate that exceeds stated budget → hard warning banner
- A simple stat block: "At your current burn rate, your cushion covers you for approximately N months without revenue."

Effort: M (4-6 hrs)

Round 3 (tax) additions

Tax as a range, not a fixed number, with a quarter/year selector

Verdict: Build it. Directly fixes a real weakness (a single "estimated income" guess presented as fact).

Have Claude return a low/mid/high band instead of one number (small prompt change), and add a simple dropdown for "which year" (current pattern already supports "quarterly" language in `quarterlyNote` — extend it to actually project 4 quarters using the Round 2 revenue growth assumptions, or the Simulator's numbers if the user ran it).

Effort: M (5-7 hrs)

Optional "will you hire employees this quarter" toggle feeding tax planning

Verdict: Build it — reuses the payroll fields from Round 2 if already entered, otherwise asks fresh.

Effort: S-M (3-5 hrs), less if payroll fields already exist from the Round 2 item above

Quarterly tax planner sub-feature (click through, enter values, get average quarterly tax)

Verdict: This overlaps heavily with the "range + quarter selector" item above — merge them.

Don't build these as two separate features. One "quarterly tax planner" panel that: shows the current quarter's estimate, lets you page through the next 3-4 quarters, and factors in the hiring toggle, covers both asks in one build.

Effort: already counted above — no separate line item.

Entity recommendation (sole prop / LLC / S-corp) — you already have this

Verdict: Already built. The tax round already returns `recommendedStructure` with entity type + reasoning. What's missing per your notes:

Which state to form the LLC in, for minimum taxes

Verdict: Build carefully — this is the one place "confidently wrong" actually hurts users.

State tax/formation-fee comparison is a real, factual question with real answers (Delaware/Wyoming/Nevada come up a lot for reasons that often don't apply to a small local business — that's a common misconception worth correcting, not reinforcing). Use the existing web-search-backed Claude call, require a cited source, and — importantly — bias the prompt toward "usually your home state is right unless you have a specific reason

not to," since that's the responsible default answer for a first-time founder, not "always pick the tax-haven state."

Effort: S-M (3-4 hrs) since it slots into the existing tax endpoint's search-enabled call.

Make the tax tab more readable

Verdict: Do this regardless of what else you build — it's currently your weakest-looking screen.

Concrete ideas, roughly in order of effort:

- 1 **Group into visual sections with icons/headers**, not one long scroll of text blocks — "Your Numbers," "Strategies," "Structure," "Next Steps" as distinct cards.
- 2 **Turn the Schedule C table into a simple horizontal bar chart** (income vs. expenses vs. net) above the detailed table — most people want the picture before the spreadsheet.
- 3 **Collapse the categorized expense table** behind a "show all N expense line items" toggle — show top 5 by default.
- 4 **Traffic-light color coding**: net profit in green/red, risk flags in amber — you already have a color system (green/amber/red) from Round 1, reuse it here.
- 5 **Turn the wall of strategy bullet points into a checklist** the user can tick off, not just read.

Effort: M (4-6 hrs), mostly CSS/layout, no backend changes.

PDF download should be instant, no prompts

Verdict: Easy fix, do it.

Right now "View" opens a print-styled page and relies on the browser's print dialog. A true one-click PDF needs a server-side PDF generation library (e.g., WeasyPrint or a headless-Chrome print-to-PDF call) so the button downloads a .pdf file directly with no dialog. This is a real, contained backend change.

Effort: M (4-6 hrs) — installing/configuring a PDF library and wiring a new download route.

New tabs proposed

Cushion Fund graph tab (with a "time commitment" sub-tab)

Verdict: Fold into the Simulator rather than making it a 6th tab.

This is the same math as the Simulator (capital, burn, runway) viewed through a different lens (how much cushion is needed, how long until profit). A hackathon demo is stronger with 4 excellent tabs than 6 thinner ones. Recommend: add "cushion fund" as a second view/toggle *within* the Simulator (bar chart of "months of runway remaining" alongside the existing line chart) rather than a standalone tab. The "time commitment" idea (how many hours/week the founder can commit) is a genuinely new input worth adding as one more Simulator field, since it affects realistic revenue ramp assumptions.

*Effort: M (5-7 hrs) if merged into the Simulator. Treat as a new **bar chart** (not a line chart) per your own note that it should be simple and readable — bar-per-month showing remaining cushion until it hits zero or turns positive is the clearest version of this.*

"Learn" tab (LLC, S-corp, cushion fund, margins, loan types, sourcing, etc.)

Verdict: Build it — and it's a strong hackathon feature for an unusual reason: it's cheap, static, and demos fast.

This doesn't need Claude calls at runtime. Write it once as a glossary/FAQ page (10-15 terms, each 2-4 sentences, in your own words or lightly Claude-assisted during build time, not live). Add a "Related sources" line per term pointing to IRS.gov/SBA.gov where relevant, consistent with your "cite real sources" theme elsewhere. A search/filter box on top makes it feel more like a product than a wiki dump. **UI ideas:** accordion-style expandable terms (not one long page), a search bar at the top, group terms under headers ("Legal structure," "Financing," "Money & margins"), and cross-link terms mentioned elsewhere in the app directly to their Learn-tab entry (e.g., the word "LLC" in the tax tab becomes a small info-icon link to the Learn tab's LLC entry) — this reuses your existing citation-tooltip pattern almost exactly.

Effort: M (5-8 hrs), almost entirely content-writing time, not engineering time.

Priority ranking if you're time-constrained

If you only have time for a subset before the hackathon, build in roughly this order — each one is chosen for the best "judge-visible impact per hour" ratio:

- 1 Tax tab readability pass (weakest current screen, all judges will open it)
- 2 Enhanced startup costs + capital-vs-budget line (Round 2) — ties budget input to a real payoff
- 3 Reality check / cushion fund fields + runway calculation
- 4 Cash flow statement + stop-loss in Simulator (cheap, uses existing data)
- 5 Tax range + quarter selector + quarterly planner (merged)
- 6 State-for-LLC recommendation
- 7 Industry margin chart
- 8 Learn tab
- 9 Debt vs. equity simplified recommendation
- 10 Cushion fund as a Simulator bar-chart view
- 11 PDF direct-download fix
- 12 Sector-based growth-rate suggestion (capped, cited version only)

Skip or defer past the hackathon: a live labor bureau API integration, and any version of the growth-rate feature that silently trusts an LLM guess without a visible source.

Rough total time estimate

Everything on the "build" list above, done efficiently in Claude-assisted sessions like this one: approximately **35-45 hours of session time** across all items, front-loaded toward the top of the priority list. Items 1-6 alone (a strong, coherent demo) are roughly **18-24 hours**.